Autonomous Robot Navigation in a Restaurant Environment

Comparison of Classical Path Planning and Reinforcement Learning with TurtleBot3 in Gazebo/ROS

Robotics Project Report

March 2026

Abstract

This report presents the design and implementation of an autonomous navigation system for a TurtleBot3 robot operating in a simulated restaurant environment. The robot must navigate from a kitchen area to five different table delivery points while avoiding obstacles. Two navigation paradigms are implemented and compared: **classical path planning** (A*, Dijkstra, and Greedy Best-First Search combined with a Dynamic Window Approach controller) and **reinforcement learning** (Proximal Policy Optimization, PPO, an Actor-Critic algorithm). The comparison evaluates path efficiency, navigation success rate, obstacle avoidance capabilities, and computation time. Results show that classical planners achieve 100% reliability with guaranteed obstacle clearance, while the RL agent produces shorter paths but with lower success rates and reduced safety margins.

Contents

1 Introduction

1.1 Context and Motivation

The deployment of autonomous mobile robots in service environments such as restaurants, hospitals, and warehouses is a growing field of research. In restaurant settings, robots must navigate safely from a preparation area (kitchen) to customer tables, avoiding static obstacles such as walls, furniture, and other structural elements.

1.2 Objectives

The primary objectives of this project are:

1. Build a simulated restaurant environment using Gazebo and ROS (Robot Operating System)
2. Implement classical path planning algorithms (A*, Dijkstra, Greedy) with a DWA controller
3. Implement a reinforcement learning agent (PPO Actor-Critic) for autonomous navigation
4. Compare both navigation paradigms on key performance metrics

1.3 System Overview

The system operates on a TurtleBot3 Burger platform simulated in Gazebo. The restaurant environment contains walls forming corridors and rooms, with 5 tables as delivery targets. The robot starts from a fixed position in the kitchen area at coordinates $(19.0, 9.0)$ in meters.

2 Environment Setup

2.1 Restaurant World Design

The restaurant environment was designed using Gazebo's SDF format. An aerial photograph of a real restaurant layout was used as reference to create the `.world` file containing:

- Walls forming corridors and rooms (modeled as box geometries)
- 5 circular tables (`cafe_table` models) positioned in the dining area
- A kitchen area where the robot spawns

2.2 Map Generation

The occupancy grid map was generated using SLAM (Simultaneous Localization and Mapping) with the `slam_toolbox` package. Initial attempts using `explore_lite` and `gmapping` failed due to wheel slippage of the TurtleBot3 on smooth surfaces. A dedicated Python script was ultimately used to capture the map reliably.

The resulting map is stored as a PGM (Portable Gray Map) file with the following parameters:

- **Resolution**: 0.05 m/pixel
- **Origin**: $(-2.0, -2.0, 0.0)$ in world coordinates
- **Occupied threshold**: 0.65
- **Free threshold**: 0.196

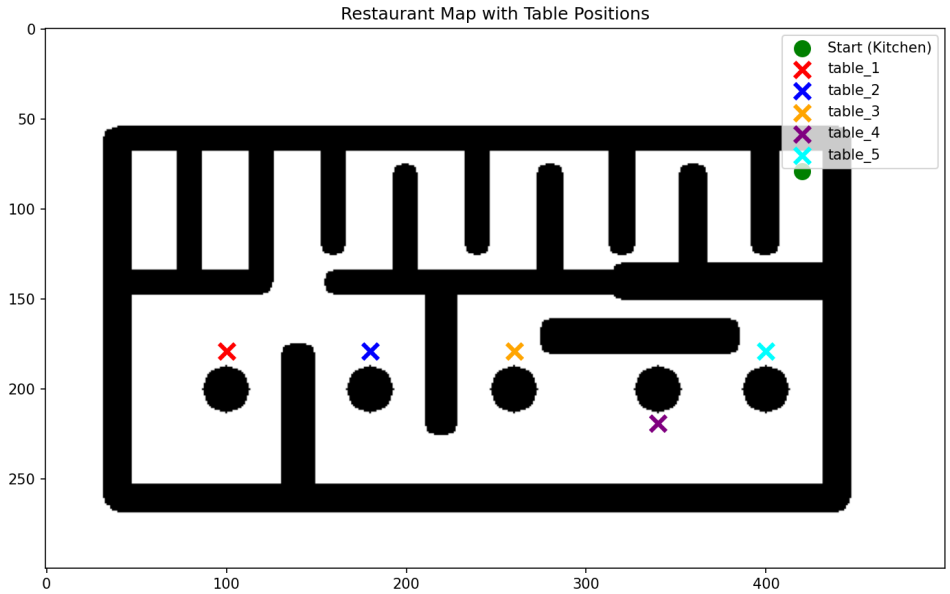

Figure 1: Restaurant occupancy grid map with the start position (green) and the 5 table delivery points.

2.3 Coordinate System and Table Positions

Table coordinates are stored in a JSON configuration file. The coordinate transformation between meters (Gazebo) and pixels (occupancy grid) is:

$$\text{col}_x = \text{round}\left(\frac{x_m - x_{\text{origin}}}{\text{resolution}}\right) \tag{1}$$

$$\text{row}_y = (H - 1) - \text{round}\left(\frac{y_m - y_{\text{origin}}}{\text{resolution}}\right) \tag{2}$$

where H is the image height in pixels, $(x_{\text{origin}}, y_{\text{origin}})$ is the map origin, and resolution = 0.05 m/pixel.

Table 1: Table delivery point coordinates (in meters)

Table ID	X (m)	Y (m)	Description
table_1	3.0	4.0	Dining area (left)
table_2	7.0	4.0	Dining area (center-left)
table_3	11.0	4.0	Dining area (center)
table_4	15.0	2.0	Dining area (center-right)
table_5	18.0	4.0	Dining area (right)

2.4 Obstacle Inflation

To ensure the robot does not collide with walls, obstacle inflation (dilation) is applied to the binary occupancy grid before pathfinding. The inflation radius is computed as:

$$r_{\text{inflation}} = \left\lceil \frac{r_{\text{robot}} + r_{\text{margin}}}{\text{resolution}} \right\rceil \text{ pixels} \tag{3}$$

With $r_{\text{robot}} = 0.20$ m and $r_{\text{margin}} = 0.05$ m, this gives $r_{\text{inflation}} = \lceil 5.0 \rceil = 5$ pixels. A circular structuring element is used with SciPy's `binary_dilation` function.

3 Classical Navigation

3.1 Path Planning Algorithms

Three classical graph-search algorithms were implemented to find paths on the inflated occupancy grid. All algorithms operate on a 4-connected grid (cardinal neighbors only).

3.1.1 A* Algorithm

A* is an informed search algorithm that combines the actual cost $g(n)$ from start to node n with a heuristic estimate $h(n)$ from n to the goal:

$$f(n) = g(n) + h(n) \tag{4}$$

The heuristic used is the Manhattan distance:

$$h(n) = |x_n - x_g| + |y_n - y_g| \tag{5}$$

A* is **optimal** and **complete** when $h(n)$ is admissible (never overestimates the true cost).

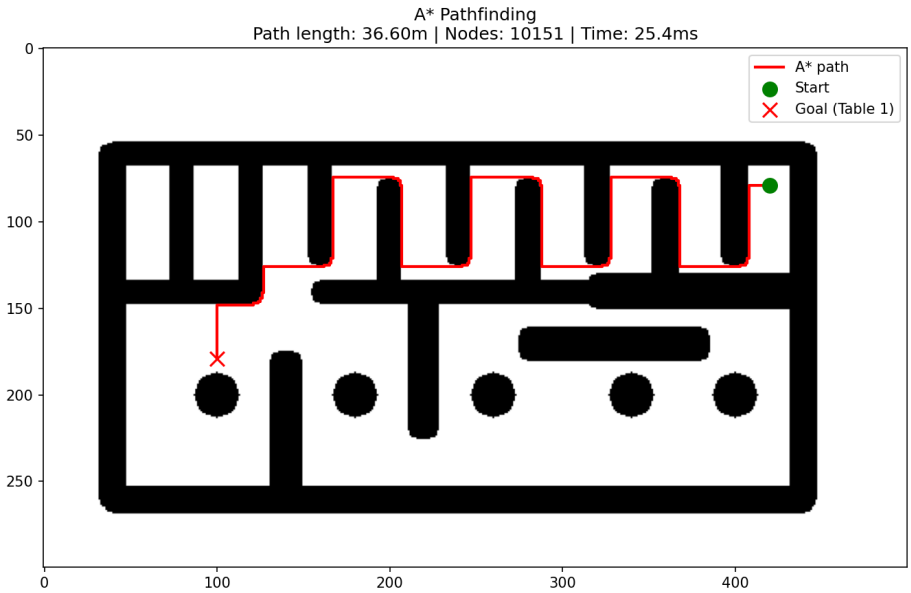

Figure 2: A* pathfinding result from the kitchen to Table 1.

3.1.2 Dijkstra's Algorithm

Dijkstra's algorithm is a special case of A* with $h(n) = 0$. It explores nodes in order of increasing cost $g(n)$:

$$f(n) = g(n) \tag{6}$$

Dijkstra guarantees the optimal path but explores significantly more nodes than A* since it has no directional guidance toward the goal.

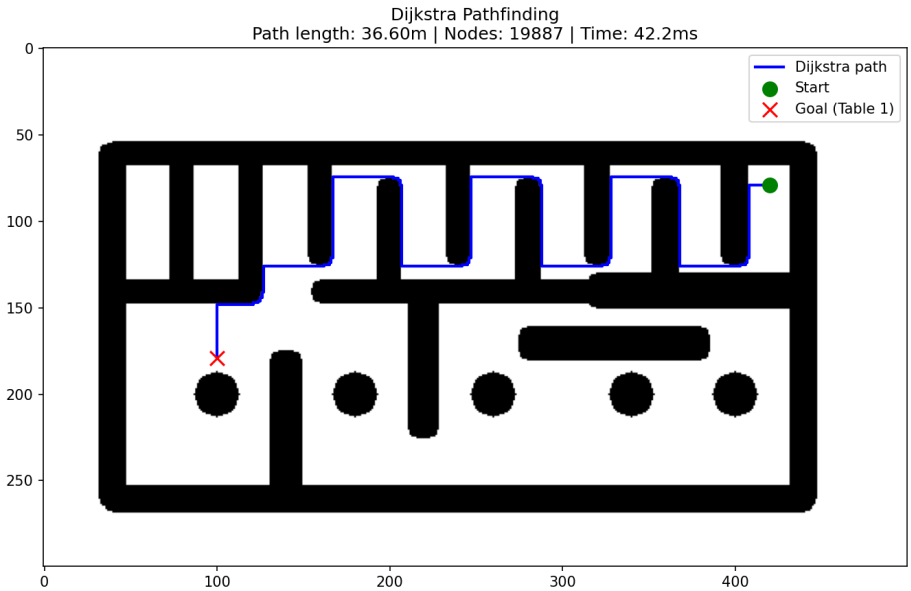

Figure 3: Dijkstra's pathfinding result from the kitchen to Table 1.

3.1.3 Greedy Best-First Search

Greedy Best-First Search uses only the heuristic to guide the search:

$$f(n) = h(n) \tag{7}$$

This makes it the fastest algorithm but **not optimal**—the path found may be longer than the shortest path. It is also not complete in the presence of certain obstacle configurations.

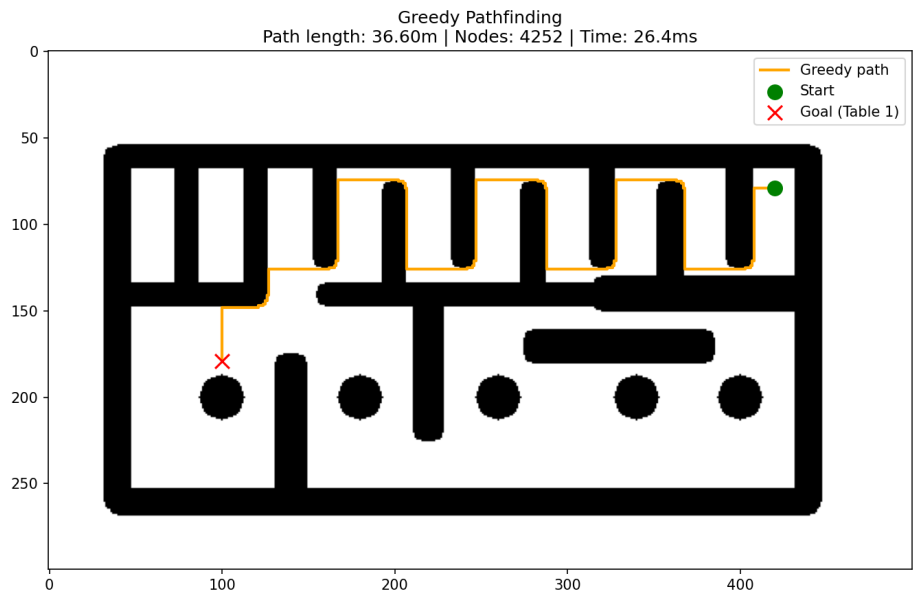

Figure 4: Greedy Best-First Search result from the kitchen to Table 1.

3.1.4 Comparison of Path Planning Algorithms

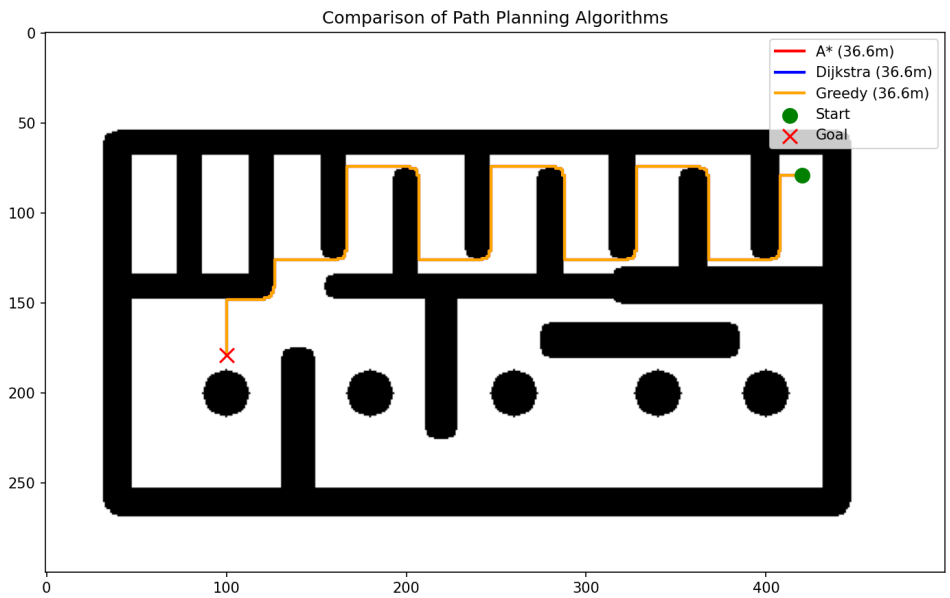

Figure 5: Overlay comparison of all three path planning algorithms on the same start-goal pair.

3.2 DWA Controller (Path Following)

Once a path is computed, the robot must follow it in the Gazebo simulation. A **Dynamic Window Approach (DWA)** controller translates the discrete grid path into continuous velocity commands (v, ω) published on the `/cmd_vel` ROS topic.

3.2.1 Kinematic Model

The TurtleBot3 is modeled as a differential-drive robot with the unicycle kinematic model:

$$\begin{cases} x_{t+1} = x_t + v\cos(\theta_t) \cdot \Delta t \\ y_{t+1} = y_t + v\sin(\theta_t) \cdot \Delta t \\ \theta_{t+1} = \theta_t + \omega \cdot \Delta t \end{cases} \tag{8}$$

where v is the linear velocity, ω is the angular velocity, and $\Delta t = 0.2$ s.

3.2.2 Dynamic Window

At each timestep, feasible velocities are constrained by the robot's acceleration limits:

$$V_s = \{(v, \omega) \mid v \in [v_c - a_v\Delta t,\ v_c + a_v\Delta t],\ \omega \in [\omega_c - a_\omega\Delta t,\ \omega_c + a_\omega\Delta t]\} \tag{9}$$

with $v_{\max} = 0.5$ m/s, $\omega_{\max} = 1.0$ rad/s, $a_v = 0.5$ m/s^2, and $a_\omega = 1.0$ rad/s^2.

3.2.3 Cost Function

Each candidate trajectory (v, ω) is evaluated over a prediction horizon of $H = 5$ steps using:

$$J(v, \omega) = \alpha \cdot d_{\text{path}} + \beta \cdot \Delta\theta_{\text{heading}} + \gamma \cdot (v_{\max} - v) \tag{10}$$

where:

- d_{path}: minimum distance from the simulated endpoint to the reference path ($\alpha = 2.0$)
- $\Delta\theta_{\text{heading}}$: angular difference to the lookahead point on the path ($\beta = 1.0$)
- $(v_{\max} - v)$: velocity reward to encourage forward motion ($\gamma = 0.5$)

The velocity pair $(v^*, \omega^*) = \arg\min J(v, \omega)$ is selected and published.

4 Reinforcement Learning Navigation

4.1 Problem Formulation as MDP

The navigation task is formulated as a Markov Decision Process (MDP):

- **State space** $\mathcal{S}$: 26-dimensional continuous vector
- **Action space** $\mathcal{A}$: 2-dimensional continuous box $[0, 0.5] \times [-1.0, 1.0]$
- **Reward function** $R(s, a)$: shaped reward (see Section **??**)
- **Discount factor** $\gamma = 0.99$

4.2 Environment: FastRobot2DEnv

A custom Gymnasium environment (`FastRobot2DEnv`) was implemented for fast training without Gazebo overhead. It uses the same occupancy grid and coordinate transformations as the classical planners.

4.2.1 Observation Space (26 values)

The observation vector consists of:

1. **Simulated LiDAR (24 rays)**: 24 distance measurements uniformly distributed over $[-\pi, \pi]$ (one ray every 15). Each ray is computed via raycasting on the occupancy grid:

$$d_i = \min\left(d_{\max},\ \min_k \left\{k \cdot \frac{\text{res}}{2} : \text{grid}\left[\text{pos} + k \cdot \frac{\text{res}}{2} \cdot \vec{u}_i\right] = \text{obstacle}\right\}\right) \tag{11}$$

   where $\vec{u}_i = (\cos(\theta + \alpha_i), \sin(\theta + \alpha_i))$ and $d_{\max} = 3.5$ m.

2. **True distance (1 value)**: The geodesic distance to the goal computed from a pre-computed **distance map** (see Section **??**).

3. **Heading angle (1 value)**: Relative angle from the robot's orientation to the goal:

$$\phi = \text{atan2}(y_g - y_r,\ x_g - x_r) - \theta_r \tag{12}$$

   normalized to $[-\pi, \pi]$.

4.2.2 Action Space

The action is a 2D continuous vector:

$$a = (v, \omega) \in [0.0, 0.5] \times [-1.0, 1.0] \tag{13}$$

where v is linear velocity (m/s) and ω is angular velocity (rad/s).

4.2.3 Distance Map (Dijkstra Flood Fill)

Instead of using Euclidean distance as reward signal (which ignores walls), a **distance map** is pre-computed for each table using a BFS/Dijkstra flood-fill from the goal position. This provides the true shortest-path distance to the goal at every grid cell:

$$D(r, c) = \min_{\text{path}} \sum_i w_i \tag{14}$$

where $w_i = 1.0$ for cardinal moves and $w_i = \sqrt{2} \approx 1.414$ for diagonal moves. Obstacle cells retain the value $D = 9999$ (unreachable).

The distance maps for all 5 tables are pre-computed once during initialization, making each `reset()` call $O(1)$ instead of $O(W \times H)$.

4.3 Reward Function

The reward function is designed to guide the agent toward the goal efficiently:

$$R(s_t, a_t) = \begin{cases} +2000 & \text{if } \|p_{\text{robot}} - p_{\text{goal}}\|_2 < 0.35 \text{ m (success)} \\ -500 & \text{if collision with obstacle (crash)} \\ \underbrace{50 \cdot (D_{t-1} - D_t)}_{\text{progress}} - \underbrace{0.5}_{\text{time penalty}} - \underbrace{0.3 \cdot |\omega_t|}_{\text{rotation penalty}} & \text{otherwise} \end{cases} \tag{15}$$

where D_t is the geodesic distance at time t (from the distance map, in meters).

Key design decisions:

- **Progress reward**: Uses the Dijkstra distance map instead of Euclidean distance, ensuring the robot is rewarded for following the actual shortest path through corridors.
- **Time penalty**: Encourages the robot to find short paths, preventing indefinite wandering.
- **Rotation penalty**: Penalizes excessive turning, producing smoother trajectories. Without this, the robot tends to oscillate (zigzag) in corridors.
- **Asymmetric terminal rewards**: The success reward (+2000) significantly exceeds the maximum cumulative penalty, ensuring the agent always prefers reaching the goal over "suiciding" into a wall.

4.4 PPO Algorithm (Actor-Critic)

Proximal Policy Optimization (PPO) is a policy gradient method that uses an Actor-Critic architecture:

- **Actor** $\pi_\theta(a|s)$: A neural network that outputs a Gaussian distribution over actions.
- **Critic** $V_\phi(s)$: A neural network that estimates the value function (expected cumulative reward from state s).

4.4.1 Policy Loss (Actor)

PPO clips the policy update to prevent catastrophically large changes:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{16}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio, $\hat{A}_t$ is the Generalized Advantage Estimate (GAE), and $\epsilon = 0.2$ is the clip range.

4.4.2 Value Loss (Critic)

$$L^{\text{VF}}(\phi) = \mathbb{E}_t \left[\left(V_\phi(s_t) - V_t^{\text{target}} \right)^2 \right] \tag{17}$$

4.4.3 Entropy Bonus

An entropy term encourages exploration:

$$H(\pi_\theta) = -\mathbb{E}[\log \pi_\theta(a|s)] \tag{18}$$

4.4.4 Total Loss

$$L(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_1 \cdot L^{\text{VF}}(\phi) - c_2 \cdot H(\pi_\theta) \tag{19}$$

with $c_1 = 0.5$ and $c_2 = 0.01$ (entropy coefficient).

4.4.5 Training Hyperparameters

Table 2: PPO Training Hyperparameters

Parameter	**Value**
Algorithm	PPO (Stable-Baselines3)
Network architecture	MLP (2 hidden layers, 64 neurons each)
Learning rate	1×10^{-4}
Rollout steps (n_{steps})	4096
Batch size	128
Clip range (ϵ)	0.2
Entropy coefficient (c_2)	0.01
Target KL divergence	0.015
Total timesteps	1,000,000
Discount factor (γ)	0.99
Episode timeout	1500 steps (150 s)
Device	CPU

4.5 Training Results

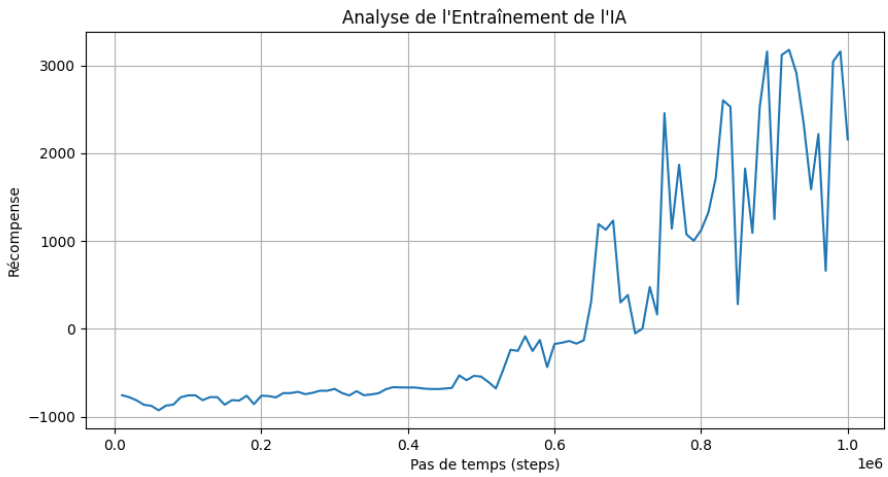

Figure 6: Training curve of the PPO agent. Mean evaluation reward over timesteps. The agent reaches stable positive rewards around ∼3000 after approximately 500,000 timesteps.

4.6 IA Navigation Trajectories

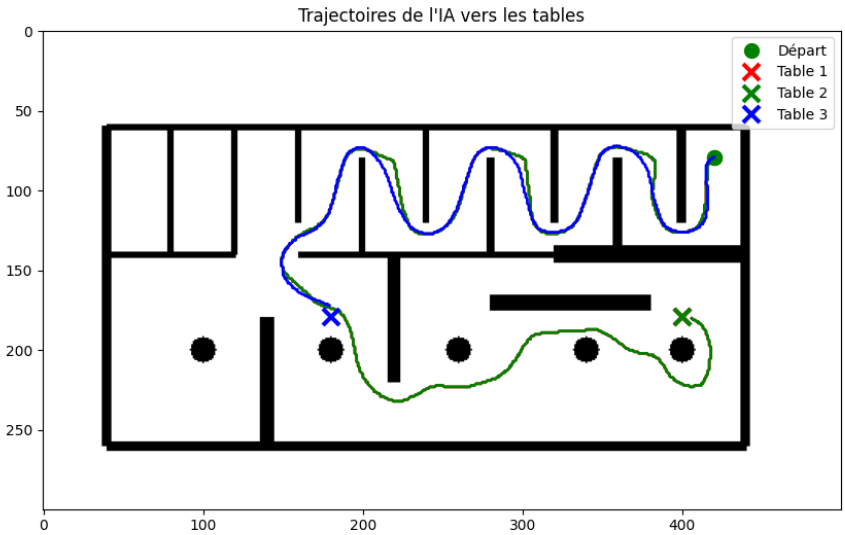

Figure 7: RL agent trajectories to different tables. The agent successfully navigates through corridors, demonstrating learned obstacle avoidance behavior.

5 Comparative Analysis

5.1 Benchmark Methodology

Both navigation paradigms were tested on all 5 tables from the same starting position $(19.0, 9.0)$. For classical planners, each algorithm was run once per table (deterministic). For the RL agent, 5 trials per table were conducted to measure variability and success rate.

5.2 Results

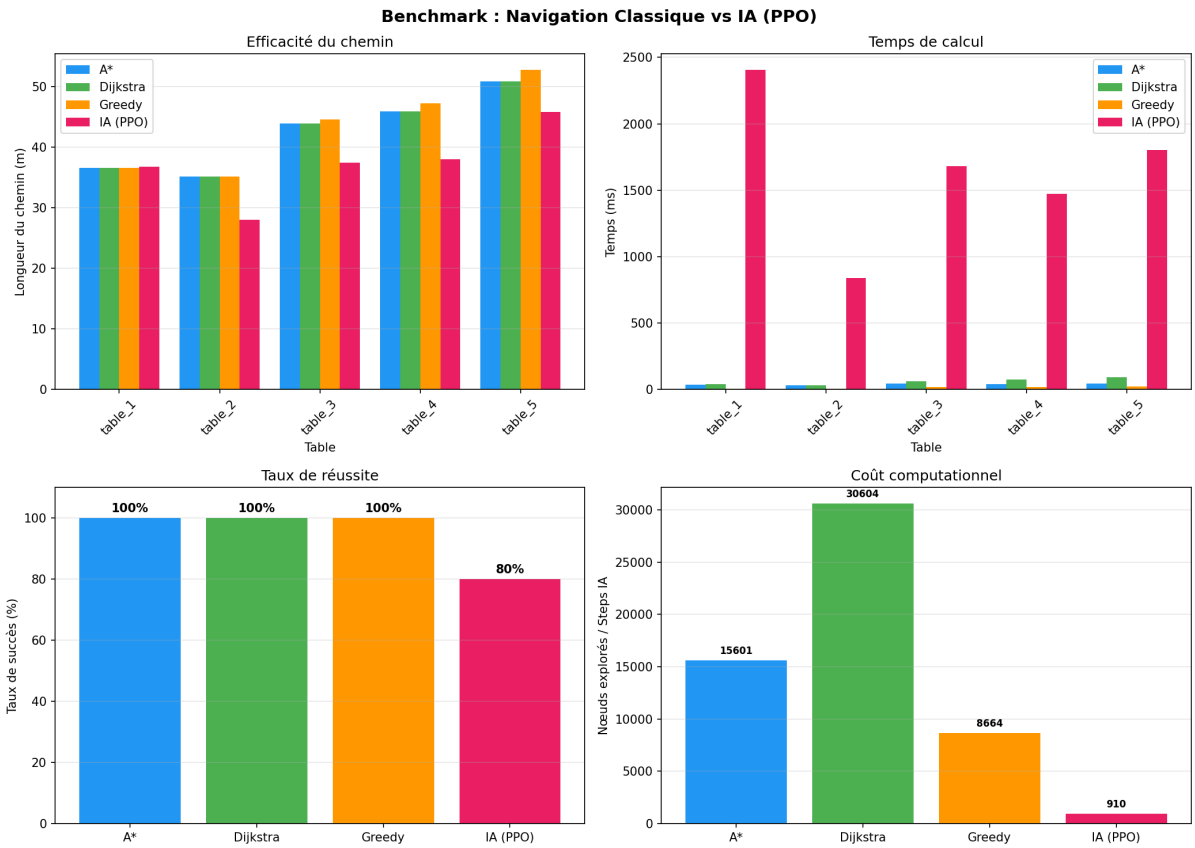

Figure 8: Comprehensive benchmark comparing classical path planning (A*, Dijkstra, Greedy) and RL navigation (PPO) across four metrics.

Table 3: Quantitative comparison of navigation approaches (averaged over all 5 tables)

Metric	A*	Dijkstra	Greedy	IA (PPO)
Success rate (%)	**100**	**100**	**100**	80
Mean path length (m)	42.50	42.50	43.30	**37.23**
Mean computation time (ms)	40.8	60.3	**15.3**	1641.4
Mean nodes explored / steps	15,601	30,604	8,664	**910**
Min. obstacle distance (m)	≥ 0.25	≥ 0.25	≥ 0.25	0.04
Crashes	0	0	0	0

5.3 Analysis

5.3.1 Path Efficiency

The RL agent produces **shorter paths** (37.23 m vs. 42.50 m for A*/Dijkstra), a 12.4% improvement. This is because the RL agent operates in continuous space and can take diagonal trajectories and smooth curves, whereas grid-based planners are restricted to 4-connected cardinal movements (up, down, left, right), introducing unnecessary Manhattan-style detours.

5.3.2 Navigation Success Rate

Classical planners achieve a **perfect 100% success rate**. They are deterministic algorithms with mathematical guarantees of completeness (A* and Dijkstra). The RL agent achieves 80% success rate, failing occasionally on certain table configurations. This is inherent to the stochastic nature of the learned policy and could be improved with additional training or curriculum learning.

5.3.3 Obstacle Avoidance

Classical planners use **obstacle inflation** (25 cm clearance), providing a hard safety guarantee. The robot *cannot* approach walls closer than this margin. The RL agent, however, navigates as close as **4 cm** to obstacles. While no crashes occurred during testing, this margin is dangerously small for real-world deployment.

5.3.4 Computation Time

Classical planners are **40–100× faster** than the RL agent at inference time. A* takes 40.8 ms, Greedy only 15.3 ms, while the RL agent requires 1641 ms per trajectory. However, this comparison excludes the RL **training time** (∼30 minutes for 1M timesteps), which is a one-time offline cost.

5.3.5 Computational Resources

Table 4: Resource requirements for each approach

Resource	**Classical (A* + DWA)**	**RL (PPO)**
Training time	None (no training)	∼30 min (offline)
Inference time per path	∼40 ms	∼1.6 s
Memory (runtime)	∼50 MB (grid)	∼200 MB (model + grid)
Dependencies	NumPy, SciPy	PyTorch, Gymnasium, SB3
GPU required	No	No (CPU sufficient)

6 Discussion

6.1 Strengths and Weaknesses

Table 5: Summary of strengths and weaknesses

Criterion	**Classical Navigation**	**RL Navigation**
Reliability	Deterministic, guaranteed	Stochastic, may fail
Path quality	Optimal (A*, Dijkstra)	Shorter but variable
Safety	Hard safety margins	Learned, no guarantee
Adaptability	Requires re-planning	Adapts via policy
Setup effort	Algorithm selection	Training + reward shaping
Generalization	Map-specific	Can generalize to new maps

6.2 Potential Improvements for the RL Agent

- **Curriculum learning**: Start training with nearby goals and progressively increase difficulty.
- **Reward shaping**: Increase the time penalty further to produce shorter paths.
- **Safety layer**: Add a hard constraint that overrides RL actions when too close to obstacles.
- **More training**: The model plateaus around 1M steps; domain randomization and longer training could improve generalization.
- **Sim-to-real transfer**: The 2D simulation does not model real-world noise, sensor inaccuracies, or wheel slippage.

7 Conclusion

This project demonstrates the implementation and comparison of two fundamentally different approaches to autonomous robot navigation in a restaurant environment. Classical path planning with DWA control provides a **reliable, fast, and safe** solution that is immediately deployable. Reinforcement learning using PPO Actor-Critic produces **shorter, smoother paths** but at the cost of reduced reliability and safety margins.

For a production restaurant robot, a **hybrid approach** would be ideal: use classical planning for path computation (guaranteed safety) and RL for local control refinement (smoother trajectories). The RL agent's ability to produce continuous velocity commands naturally complements the discrete grid paths produced by classical planners.

References

[1] Schulman, J., et al. "Proximal Policy Optimization Algorithms." *arXiv:1707.06347*, 2017.

[2] Hart, P.E., Nilsson, N.J., Raphael, B. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." *IEEE Trans. SSC*, 1968.

[3] Fox, D., Burgard, W., Thrun, S. "The Dynamic Window Approach to Collision Avoidance." *IEEE Robotics & Automation Magazine*, 1997.

[4] Raffin, A., et al. "Stable-Baselines3: Reliable Reinforcement Learning Implementations." *JMLR*, 2021.

[5] ROBOTIS. *TurtleBot3 e-Manual.* `https://emanual.robotis.com/docs/en/platform/turtlebot3/`